Python-Notes

Data types:

→ it tells to python what kind of value stored in variable

→ python need to know data type because which operation perfo-med between operands

Why do data types exist?

→ Let say we have value 10, we don't know whether it is string or number

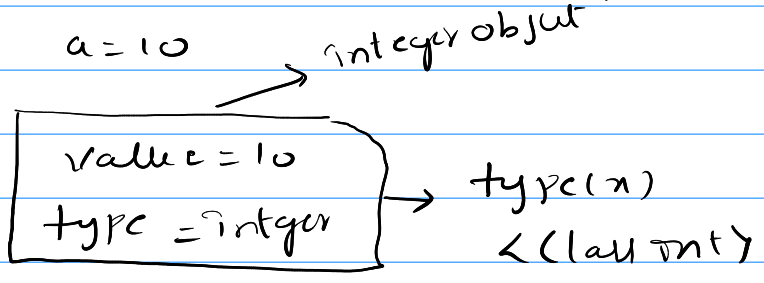
→ if we use '+' operator, then for string it is concatenation for number it is addition

```
#for numbers
a = 10
b = 10
print(a+b) # addition
#result = 30
```

```
# for strings
a = "10"
b = "30"
Print(a+b) #concate
#result = "1030"
```

→ Because of data types we know what kind of operation we need to in between variable.

* python treats everything is a object. each value associated with specific data types



* Data type helps interpreter how to store and process different kind Data efficiently

Ex : a = 10
↳ value = 10
type = int

→ Because of it is integer we can perform all kind of arthematic operations

→ Same for all the Data types with respect to their methods and operations.

Types of Data types :

Numeric Data Types :

→ Numeric data type is used to store the numeric values
→ it can be integer, float and complex

integer :- whole numbers that can negetive and positive (without fractions and decimals)

float :- it is specified by a decimal point (floating point Representation) (it is real number)

Complex :- represented by complex class, which can store both real and imagenry parts

for example :-

a = 2+5i ⟶ iota ($\sqrt{-1}$)

Real number imagenary number

example :

```
a = 5
b = 6.0
c = 2+3j
print(type(a))      # <class int>
print(type(b))      # <class float>
print(type(c))      # <class complex>
```

Sequence Data types :

→ Collection of elements of items arranged one after another

```
        0  1  2
a = [10,20,30]
```

→ Since each position has the index, we can access it via index

* 1. List
2. Tuple
3. range

+ String (Text type this is also sequence Data type)

List :

→ Lists is used store multiple values in a single variable and it is built in Data structure

→ List items are ordered, changeable, and allow duplicates and this are indexable

Positive indexing starts from 0 to n

Negative indexing starts from -n to -1

Ordered :

→ The elements have defined order, that will not change

→ if you add new items to the list, that will end of the list

Note :-

→ There is some list methods that will change the order But ingeneral it was not changeble

Ex:- insert, remove etc

changeble :-

→ This means, we can change, add, remove items in the list after it has been created

Allow duplicates :-

→ Since lists are indexed, so that list can same value as an item

List Length :-

→ To determine how many items in the list we use the

len() function

#programming

```
a = [10, 20, 30, 40]
print(len(a))
# 4
```

List Items - Data types :-

→ List items can be any data type, and it can DIFFerent types to

```
a = [1, 1.0, True, 'guru']
```

Why do we need lists? :-

imagine class room of 100 students

student1 = "gurnad"

student2 = "Jagadesh"

⋮

student 100 = "xxx"

→ it become imposible to manage.

With List :-

a = ["gurunad", "Jagadeesh" . . . . "xxxx"]

→ List makes programs

→ short

→ easy to read

→ easy to maintain

→ easy to program with loops

Creating a List :-

→ it can be done several ways

1. using [ ]
2. using list() constructor
3. using repetation of value

1. Using [ ] : it creates list Directly.

l = [1, 1.0, True, "guru"]

Using list() constructor:

#Syntax

```
variable = list(iterable)
```

↳ any obj you can iterate over.

```
a = list((10,20,30))
```

→ iterably

→ list → Dict → Range

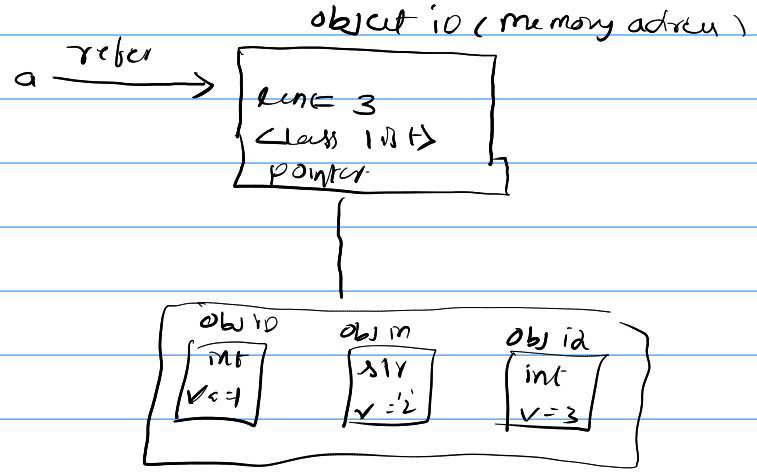
→ TUPLE → set → String

Using repetation value:-

```
a = [10]*3
print(a)
# a = [10,10,10]
```

How lists work internally:-

```
a = [1,"2",3]
```



Access List items :-

→ accessing is like retriving or showing the value. it can be Done by indexing

Indexing :-

→ a position numbu of element in a lis, it ~~always~~ starts from 0 to 1

Positive indexing 0 - n

Negative indexing -n -1

syntax of access an element

variable = my-list[index]

→ positive indexing

suppose (100) → memory adrss [start] from

l = [1, 2.0 "guru", True] → [100, 108, 106, 124)

a = l[0] → 100 ← (0x8)

print(a) → 100

#1 ← print(100)

→ Negative indexing :-

l = [1, 2.0, "guru", True]

a = l[-1]

print(a)

True

Slicing :-

→ exact a position or (subsequence in a sequence)

Syntax :-

sequence [start : stop : step]

- step → how many position to move
- stop before this indx
- start from this indx

```
   0  1  2  3  4
a = [10,20,30,40,50]
   -5 -4 -3 -2 -1
```

```
print(a[1:4])    # 20,30,40
print(a[:2])     # 10,20
print(a[:])      # 10,20,30,40,50
print(a[-4:3])   # 20,30,40
print(a[::-1])   # 50,40,30,20,10
print(a[::-2]    # 50,30,10
print(a[-4::-1:-2])   20,40
```

Update (Change the list items):

```
numbers = [100,200,300]
numbers[1] = 100
print(numbers) #
```

→ Suppose → we have addresses as a,b,c → think this as memory location

→ When you insert 100 → Then think this ID →

```
numbers = [a,b,c]
numbers = [a,D,c]
```

→ nothing happend only B → D changed.

```
a = [10,20,30]
print(id(a)); print(id(a[1]))
a[1] = 10
print(a)      # 10,20,30
print(id(a)
print(id(a[1]))
```

→ These Both are Diffrent

This Both are same

→ since integers immutable, we are not update obj ID 20, we are creating on another obj ID 10 and store, later 20 → obj ID take GC (Garbage Collector)

Change multiple items:

```
a = [10,20,30,40,50]
a[1:4] = [200,300,400]
print(a)        # [10,200,300,400,50]
```

→ Think of it like this

a = A B C D E

a = A F G H E

0 | 1 2 3 | 4 ← 1:4

→ The reference of index got replace new obj ids are created.

Replacing with Different size:

```
a = [10,20,30,40,50]
a[1:3] = [99]
print(a)    # [10,99,40,50]
```

→ Before → 5 elements → After → 4 elements

→ Python shift remaining reference left

→ memory

10 20 30 40 50

10 99 40 50

→ The list shrinks

Expanding:

```
a = [10,20,30]
numbers[1:2] = [100,200,300]
print(a)    [10,100,200,300]
```

→ the list become larger

→ python allocates additional slots to list

Updating Nest List:




```
a = [[1,2],[3,4]]
a[0][1] = 99
print(a)  # [[1,99],[3,4]]
```

Adding List items:-

→ When we say add an item we mean placing new reference in list

→ Most people confuse here

a = [10,20,30] ×

a = |10|20|30| ×

a = |1000|1008|1016| → obj ids are stored

append() :- We can add the element after last position of the list

Syntax

```
list.append(element)
```

└ element can be str, int, obj etc

```
a = [10,20,30]
a.append(40)
print(a)
```

→ Many people thought that append() → create new list add ⋯ But (NO).

→ Python allocate extra space → for some addition

→ The time complixity of append() → O(1)

→ if resizing happend then time complexity is O(n)

→ TO check the resize we have getsizeof from sys

```
import sys
a = [10,20,30,40]
print(sys.getsizeof(a))      88
a.append(50)
print(sys.getsizeof(a))      104
```

insert():-

→ whenever you want insert a element in specific indx we need Use this

syntax:-

```
list.insert(indx,value)
```

→ it insert an element before specified indxp

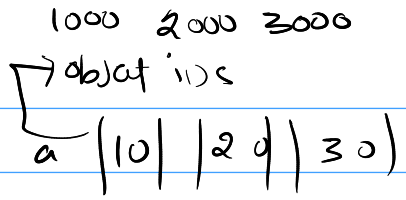
Code:

```
a = [22,32,42]
a.insert(1,31)
print(a)
#[22,31,32,37,2]
```

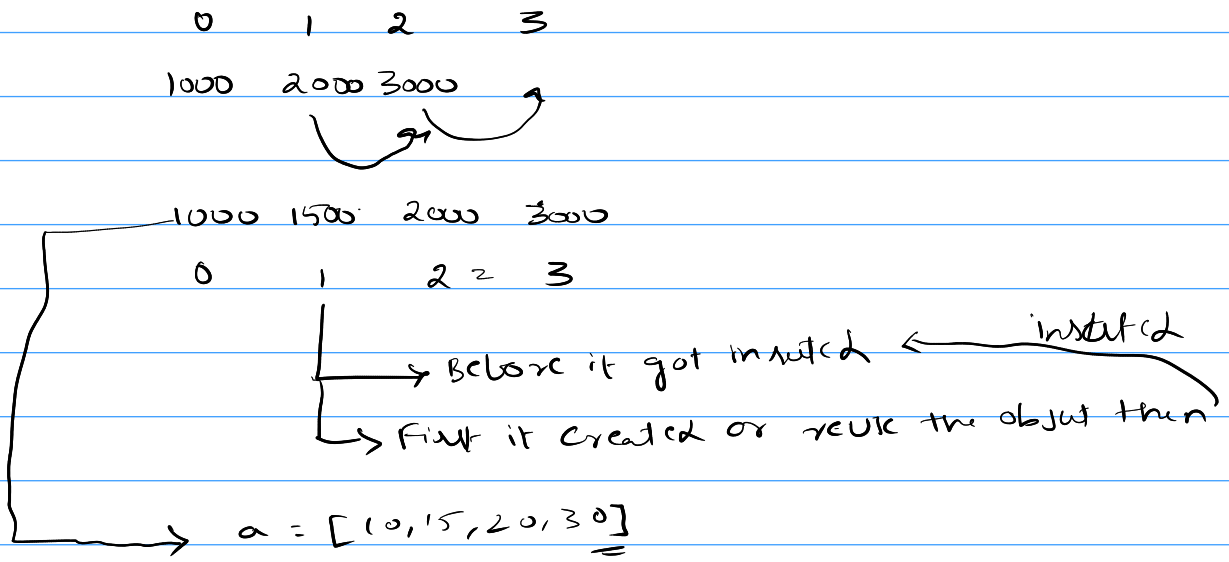
Memory level implementation

```
a = [10,20,30]
```

Now we need insert the 15 at position 1

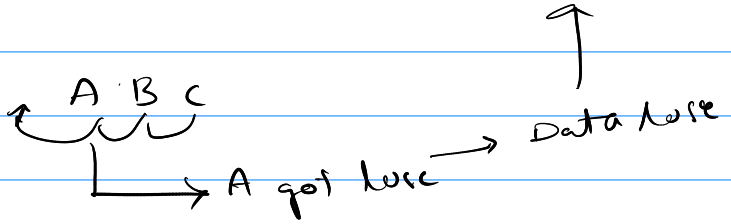


→ Now whence we need to add (insert) element (1)



→ it always should be Left to right

→ why not right to Left ? → Because of Data lose



→ if capacity is full then the Resizing happends

→ create more memory

* insert() problems

1. check capacity
2. Resize if necessary
3. Create or reuse the new obj
4. Shift refs from Left to right
5. store the new refs
6. Increase the size of list

Time Complexity :-

→ insert at the end → O(1) almost same to append()

→ Both first and middle → O(n) as the time complexity.

Code :-

```
a = [10,20,30]
for i in a:
    print(id(i))

a.insert(1,15):
for i in a:
    print(id(i))
```

Extend() :-

→ To append elements from another list to current list (not int) any kind of iterable.

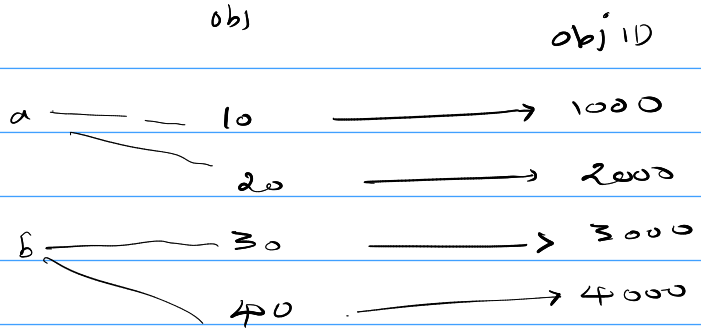
Sytax :-

```
List.extend(iterable)
```

→ it add every element of another iterable (list, tuple, Dict, set, str)
to the end of list

Code :-

```
a = [10, 20]
b = [30,40]
a.extend(b)
print(a)    #[10,20,30,40]
```



→ A only changed, b did not changed, and we copy objects from the B and add last of list

→ 1000, 2000, 3000, 4000

* Tim complexity :

O(no. of element)

↳ needs to added.

→ a is only change Beacuse copy address from b and add last of list a
→ b remains unchanged.
→ it never creat new obj or new list of the extended list the size got increased.

Code :-

```
a = [10, 20]
print(a)
for i in a:
    print(id(i))

b = [30, 40]
for i in b:
    print(id(i))

a.extend(b)
print("extend id", id(a))
for i in a:
    print(id(i))
```

Remove List items

① Remove():

→ Remove() method removes first occurrence of the specific value from a list

Syntax

```
list.remove(value)
```

→ it remove by value, not by index
→ if the value is not present Raise value error

Code

```
a = [10,20,30,40,20]
a.remove(20)
print(a)        # [10,30,40,20]
```

Memory

a → 10 | 20 | 30 | 40 | 20 → obj size = 5

after remove a's → 10 | | 30 | 40 | 20

→ It move towards Left

10 | 30 | 40 | 20 | unused | → object size = 4

→ The Time complexity of this

O(n)

→ Because shifts and search of value take O(n)

Pop():-

→ Removes an element in the list using index

→ Returning the removed element

Syntax:-

```
List.pop(index)
```

→ if no index given python removes the last element with index

Code:

```
a = [10,20,30,40]
value = a.pop()
print(value) # 40
print(a)     # [10,20,30]
```

Pop by index

```
a = [10,20,30,40]
value = a.pop(1)
print(value)  # 1
print(a)      # [10,30,40]
```

Invalid index:-

```
a = [10,20]
a.pop(5)
print(a)
```

→ pop index out of range

Memory level understanding:-

a = [10, 20, 30, 40]

Case 1 : removing the last element

a → 10 | 20 | 30 ← unused

→ Time complexity : O(1), only object_size decrease, no shifting

Case 2 : pop by index in middle or starting of the list
→ you want pop this addr

a - 1000 2000 3000 4000

a 1000 3000 4000
→ 1000 3000 3000 4000
→ 1000 3000 4000 4000
→ 1000 3000 4000 unused

We will get Answer As [10, 30, 40]

→ The Time complexity of this O(n), remaining elements shift to the Left side

Why does pop return value?

→ unlike remove pop give you the removed object

→ Because of this only pop() was used in the STACK (LIFO)

del keyword:-

→ It is not method, it is python keyword

→ del is used to delete

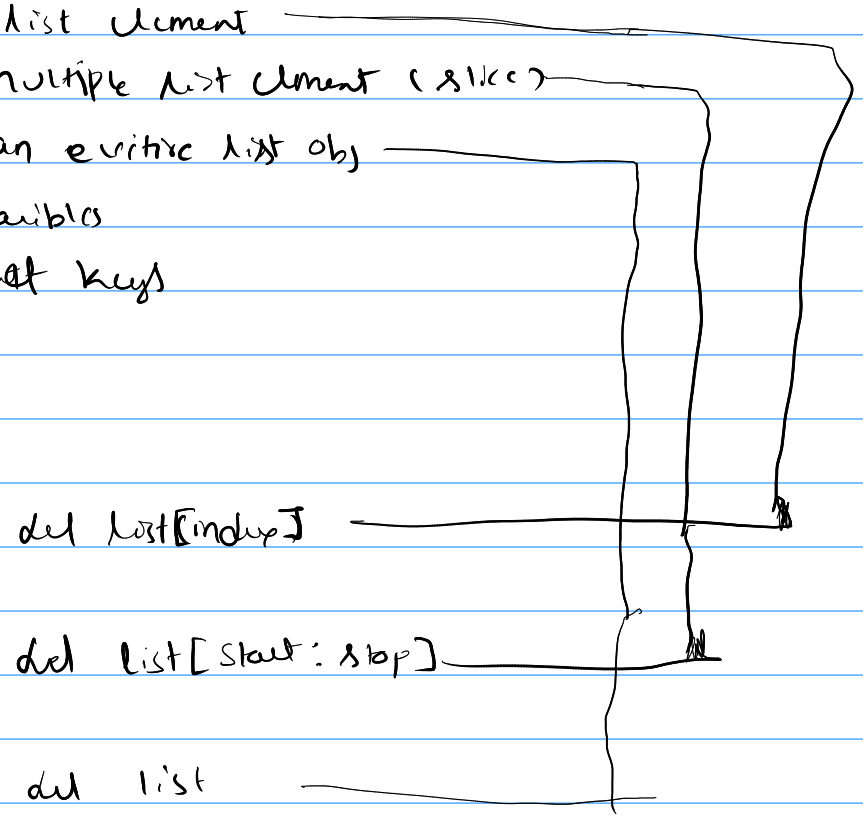
- → a list element
- → multiple list element (slice)
- → an entire list obj
- → variables
- → Dict keys

Syntax

del list[index]

del list[start:stop]

del list



Code :- Delete single element

```
a = [10,20,30,40]
del a[1]
print(a)    # [10,30,40]
```

→ it is not return any removed object

→ Memory implementation exactly like pop here

Code :- Delete slice

```
a = [10,20,30,40,50,60]
del a[1:4]
print(a)    # [10,50,60]
```

Memory Explanation :-

a → 1000 | 2000 | 3000 | 4000 | 5000 | 6000

After

a → 1000 | 5000 | 6000 | unused | unused | unused

→ Python only copies pointers → same pointer moves to left

Code :- delete entire object

```
a = [10,20,30]
del a
print(a)    # a not defined
```

DOes del delete objects:-

```
a = [10,20,30]
del a[1]
```

→ Now the pointer to 20 is removed

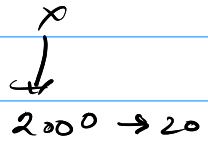
→ What about of 20?

→ That depends on reference counting

→ if no other points 20, python free it

→ If another variable still refer to it:

```
x = 20
a = [10, x, 30]
del a[1]
```



a = 1000 | 3000

→ The object 20 stays alive because x still points to it

Time complexity:-

→ del lis[-1] → O(1) → Remove the last element no shifting

→ del list[i] → O(n) → element of i shift Left

→ del lis[start:stop] → O(n) → Remaing element shift to Left

→ del list → O(1) → Remove the variable reference: obj clean up depend upon the reference count.

clear() :-

→ clear method removes all elements from the list

Syntax

list.clear()

→ After clear - The list remains exist, but it empty

Code :-

```
a = [10, 20, 30]
a.clear()
print(a)  # []
```

→ The list not deleted
→ only it content are removed

Meomory explanation :-

a = [10, 20, 30, 40, 50] → obj-size = 4, allocated = 4

→ a → 1000 | 2000 | 3000 | 4000 | 5000

→ obj-size = 0

→ a → unused | unused | unused | unused | unused

→ is clear() the same number number = [] ?

```
a = [10, 20, 30]
b = a

a.clear()
print(a)
print(b)
```

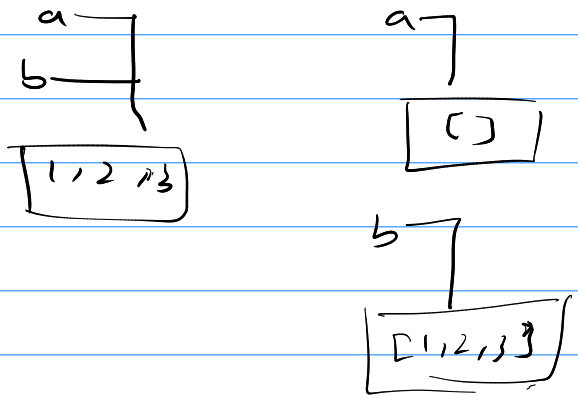
o/p :- [], []

a → 10 | 20 | 30 → now its empty []

method 2 :-

```
a = [10,20,30]
b = a
a = [ ]
print(a)    # [ ]
print(b)    # [10,20,30]
```

→ a = [ ] Creating a new empty list and make a point it
→ b still point to the original list



→ The Time complexity → O(n) → Python must remove the reference of each element and update reference count

List Comprehension :-

→ it is a shorter and efficient way to create a new list from existing iterable (string, list, tuple, Dict, Set)

→ We can write any thing using expression, instead of using multiple lines, append(), For loop etc.

Syntax

```
newlist = [expression for i in iterable]
```

↳ something that you want to store

memory execution:-

```
square = [i*i for i in range(4)]
```

initially

[ ] → [0] → [0, 1, 4, 9]

→ iteration 1 → i = 0 = 0*0 = 0

↓

internally square.append(0)

→ iteration 2 → i = 1 → 1*1 = 1

↓

internally square.append(1)

→ iteration 3 → i = 2 → 2*2 = 4

→ iteration 4 → i = 3 → 3*3 = 9

Syntax with conditions :-

[expression for variable in iterable if condition]

Ex:-

```
x = [x for x in range(20) if x>10]
print(x)
#[10 . . . . 19]
```

Nested list comprehension :-

```
matrix = [[1,2],[3,4]]
flat = [num for row in matrix for num in row]
print(flat)  # [1,2,3,4]
```

Advantages :-

→ shorter and cleaner code
→ Usually faster than an equivalent for loop, bcz Cpython uses optimized Byte code (LIST_APPEND)
→ easy to use simple transformation and filtering

Disadvantage :-

→ avoid nested complex conditions because they become difficult read

Sort List :-

→ list obj have sort() method that will sort alphanumerically ascending by default

Syntax :-

```
list.sort(reverse = True|False, key = myfun)
```

↳ True → descending, Default = False

key :- a function to specify the sorting criteria

Code :

```
a = [10, 50, 40, 20, 30]
a.sort()
print(a)   # [10, 20, 30, 40, 50]
```

Code : Alpha

```
a = ["guru", "Jaga", "Akhi", "Akhil", "ben", "banana"]
a.sort()
print(a)  # [Akhi, Akhil, ben, banana, guru, Jaga]
```

Code : Desc

```
a = [30, 40, 10, 50, 60, 20]
a.sort(reverse = True)
print(a)   # [60, 50, 40, 30, 20, 10]
```

Code : key(myfun)

```
def myfun(c):
    return len(c)

cards = ["ford", "nova", "Audi", "Bmw"]
cars.sort(key = myfun)
print(cars)      # ["Bmw", "Audi", "ford", "nova"]
```

Code : reverse()

→ What if you want to reverse the order of list

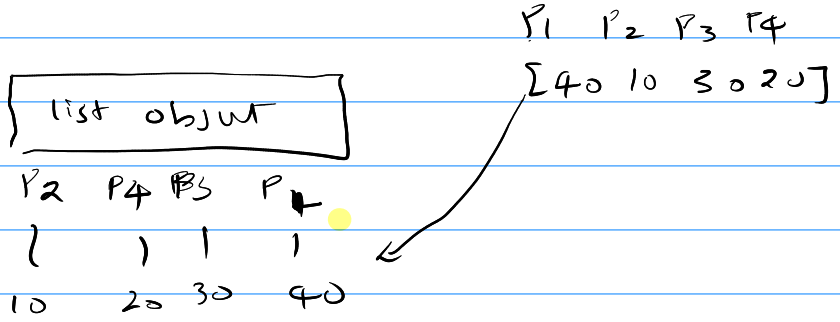
```
a = ['banana', 'orange', 'kiwi']
a.reverse()
print(a)
```

⇒ Memory level explanation :-

→ Instead it stores reference (pointers) to python objects

→ if you sort(), python did not create another list

→ same list id it never changes because python rearrange the pointers



Why sort return None ?

```
a = [3,2,1]
res = a.sort()
print(res)      # None
```

→ This is intentional, since sort() modifying existing list, returning the list again could encourage mistake so it return None

copy() :-

→ copy method create a new list object, it does not copy of element of inside list

Syntax

```
newlist = old_list.copy()
```

```
a = [10, 20, 30]
b = a.copy()
print(a); print(id(a))   # [10,20,30], 10000 } different id
print(b); print(id(b))   # [10,20,30], 20000 }
```

→ But individual element id are same like

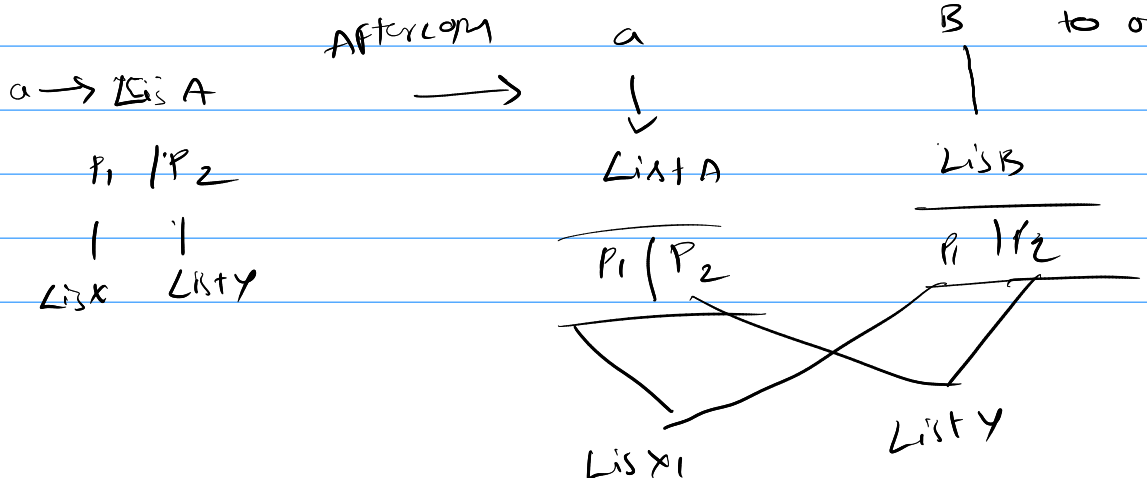
```
print(id(a[0]))
print(id(b[0]))
```
} same

→Nested List() :-

code :-

```
a = [[1,2],[3,4]]
b = a.copy()
```

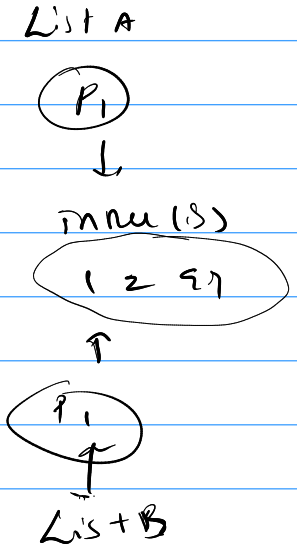
Memory → outer list is copied, inner list are not Both point to one list-



```
a = [[1,2],[3,4]]
b = a.copy()
b[0].append(99)
print(a)
print(b)
```

[[1,2,99],[3,4]]

[[1,2,99],[3,4]]



Time complexity :-

→ Python every pointer once if it has one

→ n elements then O(n)

* copy vs slicing

-> it also create a shallow copy

→ internally both have same

* list() vs copy()

b = list(a)

→ it also creates

→ a shallow copy

Join lists :-

Method 1 :- using + operator

Method 2 another one :- extend()

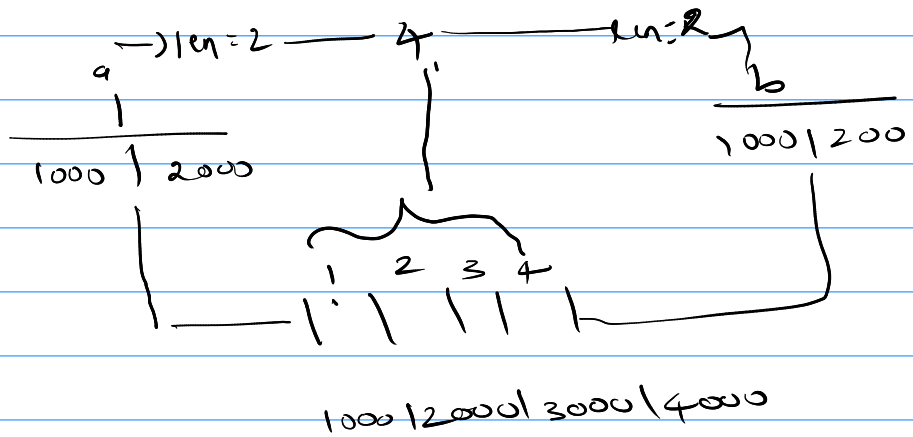
take reference from prev page.

→ when you want to two or more lists at the time we need use + operator

Code :-

```
a = [1, 2]
b = [3, 4]
c = a+b
print(c)
```

memory level :-



→ Same integer objects and 3 three Diff list objects

→ n → element in List 1

→ m → element in List 2

→ need to copy them thight so Time complexity is

O(n+m)

Tuple :-

→ it python built in data type to store multiple values (items) in single variable

→ A tuple is ordered collection of items(objects). cannot be modified after creation.

→ This is called immutability

What we can do it the tuple :-

1. Store multiple values
2. maintain insuton order
3. Allow duplicate value
4. Support index
5. Support slicing
6. Cannot add, remove, or replace elements

Why doy tuple exis?

→ Because python list which is mutable it use

→ But sometimes data never change so python introduce a data structure for this

<class tuple>

→ for example ; ADHAR ID, PAN ID, etc.

Hashable = immutable

unHathable = mutable

Len of Tuple:-

→ Len() function is used find no. of elements of a tuple

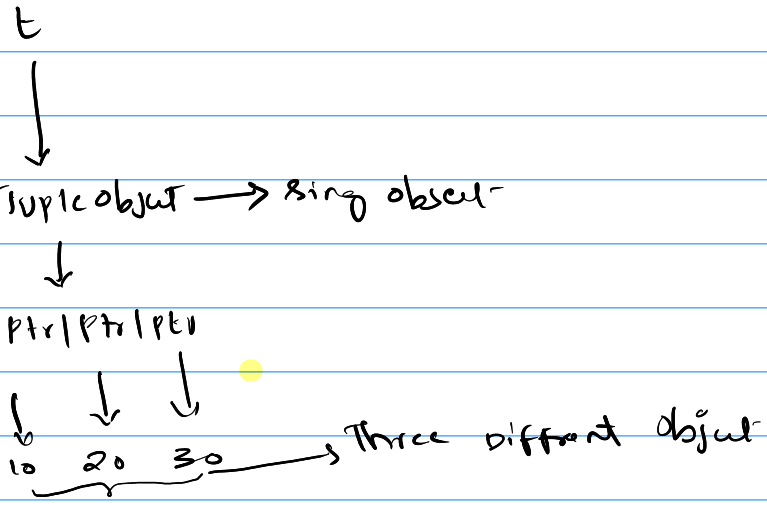
code:-

```
a = (10, 20, 30, 40)
print(len(a))  # 30
```

How python store Tuple in Memory:-

Code:-

```
t = (10, 20, 30)
```



→ The tuple doesn't directly store integers

→ It stores reference (pointers) to int objects

Does immutable mean object can't change?

Code

```
a = ([1,2], [3,4])
a[0].append(5)
print(a)    #([1,2,5], [3,4])
```

→ Here the tuple pointers never changed → P1 P2

→ and list pointers are changed what obj in

Creating tuple :-

method1 :- By using the round brackets ()

a = (10, 20, 30)

method 2 :- By using the tuple constructor

→ Syntax → a = tuple(iterable)

a = tuple([10,20,30])

Single element type :-

t = (10) → expression → Python treat this as integer

t = (10,) → , → Python treat this as tuple.

empty tuple :-

t = ()

t = tuple()

→ we can able to access through the indexing like how we access in List same here

→ and we can also check the elements present or not using in

t = ('a', 'b', 'c')

if 'a' in t:

print('yy')

→ we can add, update, remove values in tuple if you want to change anything and workaround

→ First we need to change tuple to list → Then add, remove and update the value and change it again back to the tuple

→ we can use del

Code:

```
t = (10,20,30,40)
del t
print(t)
```

Built in functions:

```
t = (10,20,30)
len()
max(t)
min(t)
sum(t)
sorted(t)
```

} return a list

tuple methods:

count() : a specific value of tuple how many time repeats

index() : it is used find the index of an element.

joining:

```
a = (1,2)
b = (3,4)
f = a+b
print(a+b)
print(f)
```

Why tuple is faster:

→ python don't need to
→ check for resizing
→ Allocate extra space
→ shift elements
→ maintain growth strategy

Hashability :-

→ Tuples can be used a DICT keys if all their element are Hashable

```
location = (10.0, 20.0)

keitys {
        } location : "Hydrabad"

print (city)    # (10.0, 20.0) : "Hydrabad"
```

→ unhashable :- TypeError

Range() :-

→ A range is a immutable sequence type that represents seq of integers without storing all integers in memory

→ instead of storing every value it stores

- → start
- → STOP
- → step

→ Then it calculate each number you asked for it

Why does range exist :-

→ if we don't have the range we need to create integer obj for every thing

```
a = [100, ..... 1000 #]
```

| 100 | 200 | 5001 . . . . 1100000 |

→ Let say we need billion integers then it takes enormous memory

→ Then python creator

" Why store every integer if we already know the patteren "

→ So they created range

→ instead of storing 0, 1, 2, 3 . . . . series

→ python stores start = 0 stop = 100000 step = 1

monemory representation:

code:

```
a = range(10)
print(a)        # 0,1,2,3,4,5,6,7,8,9
```

many people think like this

0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

But actually this how it should happend

a

start = 0
Stop = 10
step = 1

→ Only three number are stored no sequence is stored.

Why range so efficient ? :

→ Because it's Lazy

→ like it Don't create something that someone asks for it

```
r = range(5)        0,1,2,3,4

r
 start = 0
 stop = 5
 step = 1   → when py run print(r[2])

start + index * step

0 + 2 * 1 = 2
```

→ it does not store stored number, bcz they are not stored.

Range is immutable ( ) :

```
r = range(5)
r[0] = 100   # Type error
```

→ Bcz range never store a element, it store only mathematical rule

Different Ways to create :

Method 1 :- stop

```
range(stop)

r = range(5)   → means [ start = 0
                         stop = 5
                         step = 1

print(r)
# 0,1,2,3,4
```

method 2 :- start - stop

```
r = range(start, stop)
r = range(3, 8)
print(r)  # 3,4,5,6,7
```

method 3 :- start, stop, step

range (start, stop, step)

```
r = range(2, 20, 3)
print(r)   2 5 8 11 14 17
```

Code :- Negative step

```
r = range(10, 0, -2)
print(r)   10, 8, 6, 4, 2
```

→ step cannot be zero(we get error range() must not be zero)

→ We can able to use the index and slicing on the Range

Membership testing :-

5 in range(10) → By this not check every num instead

start <= number < stop

0 <= 5 < 10

(and)

(number - start) % step == 0

(5 - 0) % 1 == 0 ✓

True then True

iteration:-

```
for i in range(5)
    print(i)
```

→ The iterator keep tracking the current position

iteration - 1

current = 0

↓

print(0)

↓

current = 1

↓

print - 1

↓

current = 1

print - 2

↓

print 3

↓

print 4

↓

stop

id demonstration:-

```
r = range(5)
print(id(r))
# ID of memory location
```

converting a list

```
r = range(5)
l = list(r)
print(l)
# [0,1,2,3,4]
```

Why should you use range?

→ Looping fixed no of times

→ Generate sequence of integers

→ iterating over indexes

→ memory efficient large numbers

→ Avoid conversion of list, unless you actually need all the values stored at once

e.g

Text type:-

→ a text data type in python is string. which is for the textual information

String:-

→ String is a sequence of characters which is in single or double quotes.

→ it can be numbers, symbols and spaces

→ python does not have the seperate character datatype

→ a string is immutable sequence of character which store as single object in the memory

→ immutable :- cannot be changed after creation

→ sequence :- character are arranged in order

→ unicode ⇒ universal standard that assign a unique number to every character in every language

[A, ₹, ☺, 下, ま, Ω, €]

Why does string exist()?

→ image python no string data type

→ suppose you wanted to store

"gururadh"

→ without string

g = 'g'
u = 'u'
r = 'r'
u = 'u'

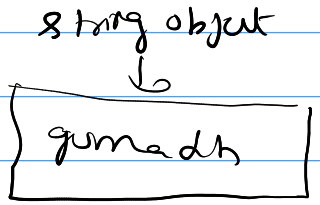
n = 'n'
a = 'a'
d = 'd'
h = 'h'

→ it would be terrible

→ instead python stores

"gurunadh"

→ inside one object



→ single character treated as the string length of 1
→ string are handling a text type and manupulation

Creating string :-

→ string can be created by either '' (or) " " quotes. Both behave the same

```
a = 'gur'
b = "guru"
print(a)
print(b)
```

→ quotes inside quotes :-

```
a = "it's okay"
b = 'my name is "guru"'
c = "my name is "guru""
print(a)
print(b)
print(c) //
```

→ created using multiline sting:
→ created using ''' ''' (or) """ """ triple quots new line is presuved.

Escaping through character:

```
Code  a = "guru"
      for i in a:
          print (a)
          print (id(i))
          print (type(i))
```

g
id
<class str>
U
id
<class str>

r
id
<class str>
U
id
<class str>

→ find length of the sting:

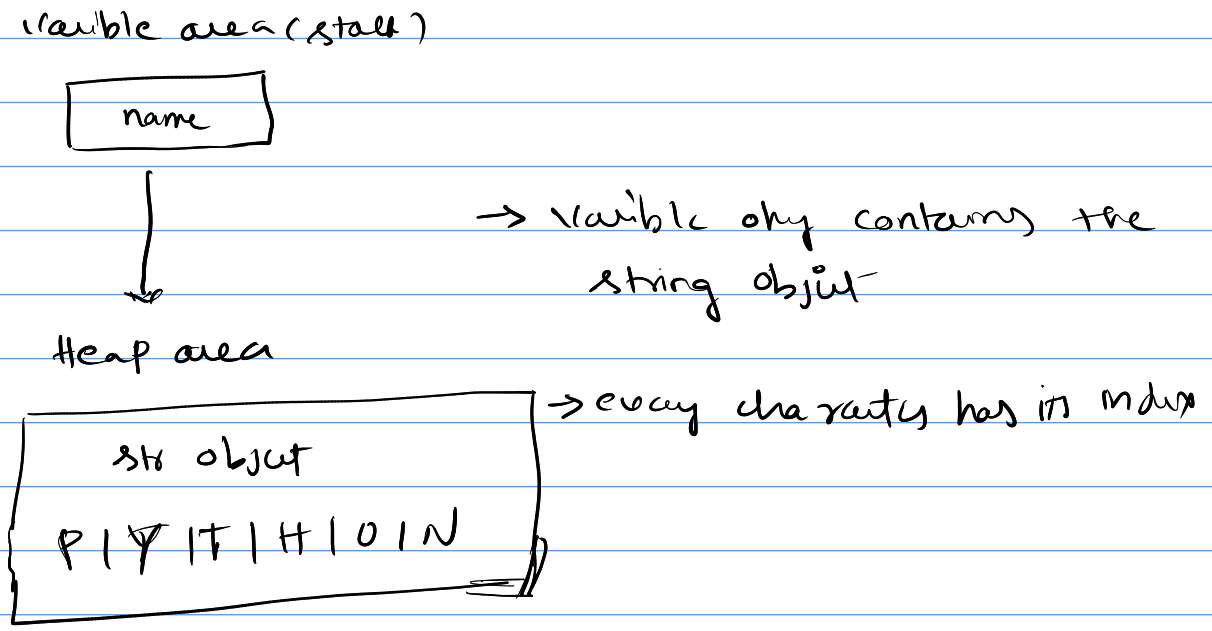
→ using lenes function we can able to find the length of the sting

```
code:   a = "guru"
        print (len(a))
        #4
```

Using membership operators:

```
Code:
        a = "my name is gurunadh"
        print('my' in a)     # true
        print ("my" not in a)  # false
```

Memory level representation:



Every character have its unicode:

→ python stores every character using unicode

Code:

```
a = 'q'
b = '@'
c = '☺'
print ord(a))
print (ord(b))
print (ord(c))
```

→ Therefore python supports any kind of characters that can be any language

String is 'immutable':

Code =

```
a = 'gurunadh'
a[0] = 'G'      error
print(a)        # str obj does not support the item assignment
```

→ Bcz python never allows changing a existing string object

Slice strings :-

→ Slicing is technique where we can get the specific parts of string using START-STOP-STEP values.

→ Specially used data manupulation and text parsing.

a = 'gurunadh'

a[START:STOP:step]

```
0 1 2 3 4 5 6 7
g u r u n a d h
-8-7-6-5-4-3-2-1
```

```
print(a[i])     # u
print(a[:5])    # gurun
print(a[5:])    # adh
print(a[1:5])   # urun
print(a[::-1])  # hdanurug
print(a[-1])    # h
```

string concatination:

→ combine two string using + operator

```
a = "guru"
b = "nadh"
Print (a+" "+b)        # guru nadh.
```

Memory level

a → has obj ID b → has obj ID

combining using +

↓

it create new object for concated string

→ notice existing won't distrub it only create new object

String repetation:

→ a string can be multiple using using *

```
a = "guru"
print (a*3)        #guruguruguru
```

→ a has a new object (old object)
→ repetation string have new object.

String len() :

→ To find the length of the string i need use the len()

len() → function

code :

```
a = "gurunadh"
print(len(a))      #8
```

Empty String :

code :

```
a = ''
print(a)
```

→ a empty string is valid object and it has some object id.

Escape character :

→ an escape character is '\' (Backslash) followed by character we want to insut

→ single quote : \'

Code :

```
a = "my name is \'gurunadh\'"
print(a) # my name is 'gurunadh'
```

→ Same for the double quote to.

→ Backslash : \\

```
a = ' my name is \\ gurnad"
print(a)   #my name is \gurnadh
```

→ New line : \n

```
a = "my name is\ngurnadh"
print(a)        #my name is
                  gurnadh
```

→ Carriage return : \r

```
a = " guru\rnadh"
gprint(a)   #~~g~~nadh
```

guru
↓
pythonscg
↓
\r
↓
drivelde
↓
guru
↓
nadh
↓
output → nadh

→ tab and back space : \t and \b

code : print("guru\tnadh") code : print("guru\bnadh")

guru ---- nadh # gurnadh

Formal Strings :-

-> f-string was introduced in python 3.6